Introduction

The **GO dataset** refers to data derived from the **Gene Ontology (GO)** project.
->not raw dataset
->knowledge-based annotation dataset
->describes **what genes do**
->where they act
->what biological processes they are involved in

What is Gene Ontology (GO)?
-> **a controlled vocabulary** that standardizes how gene functions are described across species
->What does it do?
->Where does it do it?
->Why does it do it (in which process)?

GO dataset
->Each row = one gene
->Each column = one biological function
->Each cell = whether that gene is associated with that function (1 or 0)

Binary Feature Encoding:-

`1`	This gene is associated with that GO function
`0`	This gene is NOT associated

Rows: `1124` → 1124 genes
Columns: `8642` →8640 GO features
1 gene name column
1 class label

Columns
1st-> gene name / gene identifier
GO:0006793, GO:0008150-> GO term (biological function)
Last-> CR / NotCR

`CR`	Calorie Restriction–related gene
`NotCR`	Not known to be CR-related

Overview of WorkFlow doc

2.1 Why Do We Need a Graph?
->in **table representation**, each row is independent.
->Genes do not work independently
->Genes interact, cooperate, and share similar biological functions

2.2 What Is a Graph?
->Nodes = genes
->Edges = functional similarity between genes

2.3 Nodes in Our Graph
->Every gene is represented as **one node**
->Total number of nodes = total number of genes in the dataset

2.4 Node Features
->GNNs require **features for each node**.
->Each gene has thousands of GO terms
->These GO terms describe what biological functions the gene is involved in
Feature value:
`1` → gene is associated with that GO term
`0` → gene is not associated with that GO term
->every node (gene) is represented as a **long binary feature vector**

3.1 Why Do We Need Edges?
->Edges define **how genes are connected**.
->Construct edges based on similarity between genes.

3.2 How Are Edges Constructed?
->Edges are created using GO-term similarity.
Step-by-step idea:
-Compare GO feature vectors of two genes
-Measure how similar they are
-If similarity is high, connect them with an edge

Each gene is connected to its **top-k most similar genes**
This forms a **k-nearest neighbor (kNN) graph**

3.3 What Do Edges Represent?
->Edge- two genes share similar biological roles according to Gene Ontology
->Edges **do not have direction** (undirected graph)

3.4 Node Labels (What the Model Predicts)
->Predict whether a gene is CR-related or not, using both its features and its neighbors

3.5 Final Graph Representation

After graph construction, the dataset is represented as:
->Nodes (V): genes
->Edges (E): functional similarity connections
->Node features (X): GO-term binary vectors
->Node labels (Y): CR / NotCR
This graph is then given as input to a Graph Neural Network.

3.6 Why This Graph Representation Is Useful
-Captures relationships between genes
-Allows information to flow between similar genes
-Helps the model learn patterns that are not visible in tabular data
-More suitable for biological problems where interactions matter

Implementation

Convert GO.csv dataset into a Graph
->Genes = Nodes
->Relationships = Edges
->GO features = Node attributes
->CR / NotCR = Node labels

Graph = (V, E, X, Y)
V → genes, E → similarity edges, X → GO features, Y → CR / NotCR

Node Features
Node Feature Vector = [GO:0001, GO:0002, ..., GO:8640]

Node Labels
-> CR - 1
NotCR - 0

I downloaded **9606.protein.links.v12.0.txt.gz** from STRING.
It contains [protein1 protein2 combined_score]. These are **STRING protein IDs**, NOT gene names.
So, I next downloaded **9606.protein.info.v12.0.txt.gz** from STRING.
This file maps: [ENSP ID → Gene Symbol]. To match PPI to our GO dataset.

Explanation of Graph Construction Code

1. The GO dataset is loaded and each gene is treated as a node in the graph.

2. GO term columns are extracted to form high-dimensional binary feature vectors for each gene.

3. Class labels (CR and NotCR) are encoded into numerical values for learning tasks.

4. Functional similarity between genes is computed using cosine similarity on GO features.

5. A k-nearest neighbor approach is used to connect each gene to its most similar genes, forming graph edges.

6. An undirected graph is constructed where nodes represent genes and edges represent functional similarity.

7. Node labels are stored as attributes and used to distinguish CR and NotCR genes.

8. A smaller subgraph is visualized using a force-directed layout to illustrate gene relationships.

9. Loading STRING PPI Data

10. Filtering High-Confidence Interactions

11. Mapping Protein IDs to Gene Symbols

12. Filtering by GO Dataset Genes

13. Adding PPI Edges to the Graph

14. Subgraph Extraction for Visualization

15. Edge-Type Separation

16. Graph Visualization
    [🔴 Red nodes → CR genes

    🟡 Yellow nodes → NotCR genes

    🔵 Blue edges → Real PPI interactions

    ⚫ Gray edges → GO similarity edges]

Details

1. Gene Ontology (GO) Dataset

The Gene Ontology Consortium provides a standardized, structured vocabulary for describing gene functions across different species.

It organizes biological knowledge into three categories:

i. Molecular Function [What a gene does]

ii. Cellular Component [Where it acts]

iii. Biological Process [Why it acts]

1.1 Nature of the Dataset

1.2 Dataset Structure

Component	Description
Rows	1124 genes
Columns	8642 total
Column 1	Gene identifier
Columns 2–8641	GO terms
Last Column	Class label (CR / NotCR)

1.3 Feature Representation

Each gene is represented as a binary feature vector:

Xi=[GO1,GO2,…,GO8640]

Value	Meaning
1	Gene is associated with GO term
0	Not associated

Design Justification:

- Binary encoding reflects presence/absence of biological function
- Efficient for high-dimensional sparse data
- Compatible with similarity-based methods

1.4 Labels

Class	Meaning	Encoding
CR	Calorie Restriction related gene	1
NotCR	Not CR-related	0

2 Graph Definition

G=(V,E,X,Y)

- **V (Nodes)** → Genes
- **E (Edges)** → Relationships
- **X (Features)** → GO vectors
- **Y (Labels)** → CR / NotCR

3. Node Modeling

Each gene is mapped to a node.

Assumptions:

- Each gene is a distinct biological entity
- Labels are reliable ground truth

4. Node Features

Each node carries a high-dimensional binary GO vector.

Key Assumptions:

- GO annotations are accurate and sufficiently informative
- Equal importance is assigned to all GO terms

Cosine Similarity Computation

The similarity between two genes is computed using cosine similarity:

cosine_=similarity(A,B)= (A · B)/(// A // // B //)

- Measures the angle between two vectors
- Ignores magnitude and focuses on pattern similarity
- Works well for **binary + sparse** data
- Captures **shared GO terms**

Output:

- Produces an **N × N matrix**
- Each entry Sij = similarity between gene i and j

Implementation:

1.Create an **undirected graph** because biological similarity is **mutual.**

2.Create nodes where a node is a gene.

Label stored as attribute:

- 1 → CR
- 0 → NotCR

3.Edge Construction (k-Nearest Neighbors)

Each gene connects to its **k (5)** most similar genes.

Why k=5 is used :-

Balance between:

- Connectivity
- Noise reduction

Biological reasoning:

- A gene usually interacts with a limited number of functionally similar genes
- Top 5 captures strongest functional relationships

As,Dataset size = ~1100 genes and Feature dimension = ~8640 (very high)

In high-dimensional data:

- Similarity scores can be noisy
- Keeping only top few neighbors is safer

Step 1: Get similarity scores

Step 2: Sort and pick top-k

Nodes connect if $j \in$ Top-k similar genes of i

Step 3: Edge Formation

Edge = functional similarity , Weight = similarity strength

Properties of Graph:

- Undirected
- Weighted
- Sparse (controlled by k)

Advantages:

- Keeps only **important relationships**
- Avoids fully connected graph (too noisy)
- Preserves **local structure**

This transforms tabular biological data into a graph structure where:

- Nodes represent genes
- Edges represent functional similarity
- Edge weights quantify similarity strength

To visualize, we take only 50 nodes. As, visualizing everything would be too cluttered, hard to interpret. So we extract a **small representative subgraph**

Result:

- Similar/connected nodes → closer together
- Less related nodes → far apart

Important Parameters:

`node_color`

- Assigns colors based on class

`with_labels=True`

- Displays gene names
- Useful for interpretation

`node_size=900`

- Makes nodes clearly visible

`font_size=7`

- Prevents label overlap
- Keeps visualization readable

`seed=42`

- Ensures **same layout every time**
- Useful for:
    - Reproducibility
    - Comparing results

Loading and Filtering PPI Data (STRING Database)

PPI (Protein-Protein Interaction) data represents **physical or functional interactions between proteins**. Captures **real biological relationships**, unlike similarity-based edges.

- Source: **STRING database**
- Website: https://string-db.org/

Files Used:

1. **`9606.protein.links.v12.0.txt.gz`**

- Contains:
    - Protein–protein interactions
    - Confidence scores

2. **`9606.protein.info.v12.0.txt.gz`**

- Contains:
    - Mapping from protein IDs to gene symbols

Description:

- Species: **Homo sapiens (9606)**
- Interaction score: Range: 0–1000

Step1: Loading PPI Interaction Data

File Contents:

Column	**Meaning**
protein1	First protein
protein2	Second protein
combined_score	Confidence of interaction

Step2: Loading Protein Info Mapping

Maps: Protein ID → Gene name

GO dataset uses: **Gene names (symbols)**

But STRING uses:**Protein IDs**

So, without mapping, datasets cannot be merged.

Step3: Filtering High-Confidence Interactions

`Combined score` is

- Provided by STRING
- Range: **0 to 1000**

Represents confidence of interaction based on:

- Experiments
- Databases
- Co-expression
- Text mining

Threshold = 700 because

Threshold	Meaning	Effect
< 400	Low confidence	More edges but noisy
400–700	Medium	Balance between reliability and coverage
> 700	High confidence	Very reliable but sparse

Reasoning:

- Keep only **biologically reliable interactions**
- Remove noisy/uncertain edges

Output shows the number of **strong interactions retained**.These filtered interactions will become edges in the graph.

Integrating PPI Data into the Gene Graph

Step1: Mapping Protein IDs to Gene Names

Problem:

- PPI dataset uses **protein IDs (ENSP...)**
- GO dataset uses **gene names**

So, these are **different identifier systems.**

We need to create a mapping: Protein ID→Gene

Assumption:

- Each protein maps correctly to a gene
- Mapping file is accurate and complete

Step2: Applying Mapping to PPI Data

Applying Mapping to PPI Data. Some mappings may fail so, result = NaN

Step3: Removing Invalid Mappings

Reason:

- Remove interactions where mapping is missing
- Ensures data consistency

Assumption:

- Rows with missing mapping are unreliable

Step4: Filtering by GO Dataset Genes

Reasoning:

- Avoid adding unknown/unseen nodes
- Maintain same node space (V)

Assumption:

- Only interactions among GO genes are relevant

Step5: Adding PPI Edges to Graph

Each edge represents real biological interaction between proteins (genes).

Edge Attributes:

Attribute	**Meaning**
relation="PPI"	Marks edge type

weight Confidence score

We store "relation"

As, Enables multi-edge type graphs.

Useful for:

- Visualization
- GNN extensions

Step6: Graph After Adding PPI

Total edges now include:

- GO similarity edges
- PPI edges

Edge Colors:

Type	Color
GO similarity	⚫ Gray
PPI	🔵 Blue

Node Colors:

Type	Color
CR	🔴 Red
NotCR	🟡 Yellow

Observations in the graph:

Dense central cluster:

Genes with:

- High similarity
- Many interactions

Isolated nodes:

Genes that:

- Have similarity edges
- But no PPI connections in subset